

ΕΡΓΑΣΙΑ ΣΤΟΝ ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΕΦΗ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟ

Αγωνία

ΤΡΙΤΟ ΠΑΡΑΔΟΤΕΟ

Υλοποίηση συνολικής εφαρμογής

Το τρίτο παραδοτέο της εργασίας απαιτεί τη δημιουργία συναρτήσεων και την ενσωμάτωσή τους στο συνολικό κώδικα.

Σας δίνονται όλα τα αρχεία κώδικα, ενώ πρέπει να συμπληρωθούν μόνο τα Deck, Game και Main. Ακολουθούν αναλυτικές οδηγίες ανά αρχείο.

Κλάση Deck

Η κλάση Deck αναπαριστά την τράπουλα του παιχνιδιού. Η τράπουλα είναι η μοναδική κλάση που δημιουργεί αντικείμενα Card.

Μια τράπουλα έχει:

- cards (δυναμικός πίνακας δεικτών στα αντικείμενα Card που ανήκουν στην τράπουλα)
- currentSize (το πλήθος των καρτών που υπάρχουν αυτή τη στιγμή στην τράπουλα)
- capacity (η μέγιστη χωρητικότητα — 52)

Η κλάση πρέπει να υποστηρίζει:

- Συνάρτηση αρχικών συνθηκών η οποία κατασκευάζει ένα αντικείμενο τύπου Deck και αρχικοποιεί το capacity = 52, το currentSize = 0, δεσμεύει δυναμικά τον πίνακα cards και δημιουργεί και τα 52 αντικείμενα κάρτας. Για κάθε συνδυασμό χρώματος (4 χρώματα) και αξίας (1–13) δημιουργείται το αντίστοιχο αντικείμενο της σωστής υποκλάσης. (Για τη δημιουργία της τράπουλας ακολουθήστε την παρακάτω σειρά, για κάθε χρώμα-φιγούρα (με τη σειρά HEART, DIAMOND, SPADE, CLUB) δημιουργήστε κάθε φύλλο (1→13) και στη συνέχεια προχωρήστε στο επόμενο χρώμα-φιγούρα)
- Συνάρτηση τελικών συνθηκών η οποία καταστρέφει ένα αντικείμενο τύπου Deck και διαγράφει κάθε αντικείμενο Card που βρίσκεται ακόμα στην τράπουλα, και στη συνέχεια διαγράφει τον ίδιο τον πίνακα δεικτών. Προσοχή: κάρτες που έχουν ήδη μοιραστεί σε παίκτες ή βρίσκονται στο τραπέζι δεν διαγράφονται εδώ.
- reset(): διαγράφει όλες τις κάρτες που υπάρχουν ακόμα στον πίνακα, μηδενίζει το currentSize, και ξαναδημιουργεί και τα 52 αντικείμενα κάρτας. Καλείται στην αρχή κάθε νέου γύρου.
- deal(): αφαιρεί και επιστρέφει την κάρτα κορυφής (τελευταίο στοιχείο του πίνακα). Αν η τράπουλα είναι άδεια επιστρέφει nullptr.
- rebuildFromDiscard(Card** discardPile, int discardSize): όταν η τράπουλα τελειώσει στη μέση ενός γύρου, ανακυκλώνει τις κάρτες του τραπέζιου. Προσθέτει στον πίνακα cards όλες τις κάρτες του discardPile εκτός από την τελευταία (η κάρτα κορυφής παραμένει στο τραπέζι), και στη συνέχεια καλεί shuffle().

Κλάση Game

Η κλάση Game είναι ο κεντρικός ελεγκτής του παιχνιδιού. Κατέχει την τράπουλα (Deck), το τραπέζι (Table), και δείκτες στους παίκτες. Ένα παιχνίδι έχει:

- noOfPlayers (ο συνολικός αριθμός παικτών)
- deck (η τράπουλα ανάληψης)
- table (το τραπέζι — σωρός αποβολής)
- players (δυναμικός πίνακας δεικτών στους παίκτες)
- currentPlayerIndex (ο δείκτης του παίκτη που παίζει αυτή τη στιγμή)
- skipNext (αν ο επόμενος παίκτης χάνει τη σειρά του — εφέ του 9)

- `mustPlayAgain` (αν ο τρέχων παίκτης παίζει ξανά — εφέ του 8)
- `penaltyStack` (συσσωρευμένη ποινή από 7άρια)

Η κλάση πρέπει να υποστηρίζει:

- Συναρτήσεις αρχικών συνθηκών οι οποίες δέχονται σαν είσοδο τους παίκτες του παιχνιδιού. Κάθε constructor αρχικοποιεί τα πεδία (`currentPlayerIndex = 0`, `skipNext = false`, `mustPlayAgain = false`, `penaltyStack = 0`), δεσμεύει δυναμικά τον πίνακα `players` με `new Player*[noOfPlayers]`, και αποθηκεύει απευθείας τους δείκτες των παικτών στον πίνακα.
- `startRound()`: προετοιμάζει το παιχνίδι για νέο γύρο. Συγκεκριμένα:
 - Για κάθε παίκτη: διαγράφει τα αντικείμενα `Card` στο χέρι του, καλεί `clearHand()`, `setHasDrawnCard(false)`, `setStatus(HASNTPLAYED)`
 - Διαγράφει τις κάρτες του τραπέζιου και καλεί `table.clearPile()`
 - Καλεί `deck.reset()` και `deck.shuffle()`
 - Μοιράζει 7 κάρτες σε κάθε παίκτη (κάθε παίκτης “τραβάει” κάρτα)
 - Τοποθετεί 1 κάρτα στο τραπέζι με `table.addCard(deck.deal())`
 - Μηδενίζει τα flags: `penaltyStack = 0`, `skipNext = false`, `mustPlayAgain = false`, `currentPlayerIndex = 0`
- `advanceTurn()`: προχωράει στον επόμενο παίκτη, λαμβάνοντας υπόψη τα εφέ των ειδικών καρτών:
 - Αν `mustPlayAgain == true`: μηδενίζει το flag και ο ίδιος παίκτης παίζει ξανά
 - Αλλιώς: προχωράει στον επόμενο
 - Αν ολοκληρώθηκε πλήρης περιφορά: επαναφέρει τα status `PASSED`, `PLAYED`, `MISSEDTURN` σε `HASNTPLAYED`
 - Αν `skipNext == true`: τυπώνεται μήνυμα πως ο παίκτης έχασε τη σειρά του (χρησιμοποιήστε `cout << "\n" << [current_player_name] << " misses their turn!" << endl;`), λαμβάνει status `MISSEDTURN`, μηδενίζεται το `skipNext`, και προχωράει ξανά στον επόμενο
- `replenishDeckIfNeeded(int neededCards)`: αν η τράπουλα έχει λιγότερες κάρτες από `neededCards`, ανακυκλώνει τις κάρτες του τραπέζιου: κρατάει την κάρτα κορυφής (τελευταία που έχει παιχτεί), ξανά “χτίζει” την τράπουλα από τις κάρτες αυτές και καθαρίζει το τραπέζι έχοντας πάνω σε αυτό μόνο την κάρτα κορυφής.
- `applyPenalty(Player* p)`: αναγκάζει τον παίκτη `p` να τραβήξει `penaltyStack` (ο αριθμός των καρτών που πρέπει να τραβήξει με βάση τα 7άρια που έχουν παιχτεί) κάρτες. Τυπώνει μήνυμα, τραβάει τις κάρτες (χρησιμοποιήστε το εξής: `cout << [player_name] << " absorbs " << penaltyStack << " penalty card(s)\n";`), μηδενίζει το `penaltyStack`, και καλεί `setHasDrawnCard(false)` ώστε ο παίκτης να μπορεί να συνεχίσει κανονικά.

Τέλος στο αρχείο `main.cpp` καλείστε να υλοποιήσετε τις συναρτήσεις `getValidCards` και `handleChoice` οι οποίες βοηθούν ώστε ο παίκτης να μπορεί να παίζει κατά τη σειρά του.

- Η `getValidCards` επιστρέφει τις κάρτες τις οποίες μπορεί να παίξει. Προσοχή καθώς στην περίπτωση που έχει πέσει 7άρι, άρα το `penaltyStack` είναι μεγαλύτερο του 0 θα πρέπει να επιστρέφει μόνο τα 7άρια που έχει (hint δείτε τη συνάρτηση `handlePenaltyPhase` για να κατανοήσετε τι γίνεται αν έχει ή δεν έχει 7αρι στα χέρια του). Επίσης θα πρέπει να αλλάζει κατάλληλα τη μεταβλητή `outCount` που αντιστοιχεί στον αριθμό των `valid` καρτών.
- Η `handleChoice` είναι η υπεύθυνη συνάρτηση για να διαχειριστεί την επιλογή του χρήστη και να ορίσει αν ολοκληρώθηκε ο γύρος του παίκτη καθώς και αν ολοκληρώθηκε το παιχνίδι. Έτσι η συνάρτηση θα πρέπει να κάνει τα ακόλουθα:
 - Αν η επιλογή του παίκτη (`choice`) είναι -1:
 - Αν δεν έχει τραβήξει και δεν υπάρχει ποινή, τυπώνεται μήνυμα πως ο παίκτης

- τραβά μία κάρτα (χρησιμοποιήστε `cout << [current_player_name] << " draws 1 card(s).\n";`) και στη συνέχεια τραβάει 1 κάρτα (`drawCard`)
- Αν δεν έχει τραβήξει κάρτα και υπάρχει ποινή απορροφά την ποινή
 - Αν έχει ήδη τραβήξει φύλλο πάει πάσο (`pass`), αλλάζει η μεταβλητή `turnEnded` σε `true` και τυπώνεται το μήνυμα ότι πήγε πάσο (χρησιμοποιήστε `cout << [current_player_name] << " passes.\n";`)
 - Αν η επιλογή του είναι μέσα στις επιτρεπτές επιλογές:
 - Αφαιρεί την κάρτα από το χέρι ("παίζει" την κάρτα), και εφόσον αφαιρεθεί
 - Ελέγχει εάν στην εκάστοτε περίπτωση υπήρχε ενεργό `penaltyStack` και η κάρτα που έπαιξε ο παίκτης είναι 7 ώστε να τυπώσει το κατάλληλο μήνυμα ότι απέκρουσε το `penalty` (χρησιμοποιήστε `cout << [current_player_name] << " successfully bounces the penalty with another 7!\n";`)
 - Προσθέτει την κάρτα στο τραπέζι
 - Καθαρίζει το δηλωμένο `suit`
 - Εφαρμόζει το `effect` της κάρτας που έπεσε και τυπώνει το αντίστοιχο μήνυμα(χρησιμοποιήστε τα παρακάτω
`cout << [current_player_name] << " raised the penalty stack!\n";`
`cout << [current_player_name] << " gets to play another card!\n";`
`cout << "Nine played! Next player loses their turn.\n";`
`cout << [current_player_name] << " declares suit: " << suitToString(newSuit) << "\n";`). Προσοχή στην περίπτωση του άσου πρέπει να οριστεί και `suit`.
 - Ελέγχει εάν ο παίκτης δεν έχει άλλες κάρτες στο χέρι ώστε να τυπώσει το κατάλληλο μήνυμα (χρησιμοποιήστε `cout << [current_player_name] << " has no more cards. Round ends!\n";`) και να θέσει την τιμή της μεταβλητής `gameRunning` σε `false`.
 - Θέτει τη μεταβλητή `turnEnded` σε `true`
 - Εάν δεν αφαιρεθεί για κάποιο λόγο η κάρτα από το χέρι τυπώνεται το αντίστοιχο μήνυμα (χρησιμοποιήστε `cout << "Error: Could not play the card. Please try again.\n";`)
 - Εάν η επιλογή δεν είναι μέσα στις επιτρεπτές θα πρέπει να τυπώνεται αντίστοιχο μήνυμα (χρησιμοποιήστε `cout << "Invalid choice. Please pick a number from 0 to " << validCount - 1 << " or -1.\n";`)

Σας δίνεται ένα αρχείο `zip` όπου περιέχει το σύνολο του κώδικα χωρίς τις συναρτήσεις που πρέπει να υλοποιήσετε. Μπορείτε να το χρησιμοποιήσετε για να κάνετε δοκιμές, να δείτε πως είναι υλοποιημένες οι λοιπές συναρτήσεις και στο τέλος θα πρέπει να υποβάλετε τον κώδικά σας στα αντίστοιχα σημεία του `quiz` της κατάθεσης της εργασίας

Παραδοτέο

Το παραδοτέο της εργασίας είναι η υποβολή του αντίστοιχου `quiz` στο οποίο θα πρέπει να γίνει η υλοποίηση των αντίστοιχων κλάσεων και συναρτήσεων.

Προθεσμία υποβολής

Το παραδοτέο πρέπει να παραδοθεί μέχρι τις 23:59 της Κυριακής 28 Ιουνίου. Καμία παρέκκλιση δεν θα γίνει από την παραπάνω προθεσμία.